Git local, modelo mental, Markdown y primeras ramas

Introducción

En esta sesión vas a construir la base real de trabajo con Git. No se trata solo de aprender comandos, sino de entender qué espacios existen dentro del flujo de versionado, cómo se mueve la información entre ellos y por qué Git te obliga a tomar decisiones conscientes antes de registrar cambios.

También empezarás a trabajar con `README.md` y con Markdown, porque en proyectos reales Git no se utiliza únicamente para código: también organiza documentación, notas técnicas, instrucciones de uso y comunicación del proyecto. Por eso tiene sentido comenzar con archivos de texto y documentación antes de pasar a flujos más complejos.

Además de dominar el trabajo local, en esta sesión vas a introducir las primeras ramas, verás la diferencia entre `switch` y `checkout`, empezarás a usar mensajes de commit con intención y conocerás herramientas muy habituales como `commit --amend`, `commit -am`, `stash` y `alias`.

El objetivo final es que puedas explicar con claridad la diferencia entre tu espacio de trabajo, el área de preparación, el historial local, el almacenamiento temporal de `stash` y la futura relación con el repositorio remoto.

---

1. Qué es Git y qué problema resuelve

Git es un sistema de **control de versiones distribuido**. Su función principal es registrar la evolución de un proyecto a lo largo del tiempo, permitiéndote conservar estados, recuperar versiones anteriores, comparar cambios y organizar el desarrollo de forma trazable.

Sin control de versiones, es fácil caer en prácticas poco sostenibles:

- duplicar carpetas con nombres como `proyecto_final`, `proyecto_final_bueno` o `proyecto_final_definitivo`;
- sobrescribir archivos sin posibilidad de recuperación razonable;
- perder cambios tras una modificación errónea;
- no saber qué se cambió, cuándo se cambió o por qué se cambió.

Git resuelve estos problemas mediante un historial estructurado y navegable.

1.1. Qué te aporta Git en la práctica

Git te permite:

- registrar cambios de forma incremental;
- recuperar estados anteriores de un archivo o de un proyecto completo;
- comparar versiones;
- separar trabajo en ramas;
- documentar la evolución del proyecto;
- preparar colaboración futura con otras personas;
- construir un historial técnico útil.

1.2. Git no es una copia de seguridad automática

Un error muy frecuente al empezar es imaginar que Git registra automáticamente todo lo que haces en cuanto editas un archivo. No funciona así.

Git te obliga a decidir:

- qué cambios quieres preparar;
- qué cambios quieres confirmar;
- cuándo quieres incorporarlos al historial.

Eso hace que Git no sea solo un sistema para guardar archivos, sino una herramienta para construir versiones con intención.

> **Idea clave**
>
> Git no registra tu trabajo de forma automática. Tú decides qué entra en el historial y en qué momento lo hace.

Referencias del subtema

- Git Book, introducción al control de versiones.
- Documentación oficial de Git.
- Explain Git with D3.

---

2. Instalación y configuración inicial

Antes de usar Git con normalidad, necesitas verificar que está instalado y configurar tu identidad como autor o autora de los commits.

2.1. Comprobar la instalación

En terminal, puedes comprobar la instalación con:

```
git --version
```

Si Git está correctamente instalado, verás la versión disponible en tu sistema.

2.2. Configurar tu identidad

Git asocia cada commit a una identidad. Para configurar la tuya de forma global, puedes usar:

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu_correo@example.com"
```

La opción `--global` indica que esta configuración se aplicará por defecto a todos los repositorios de tu usuario en ese equipo.

2.3. Consultar la configuración existente

Puedes revisar la configuración activa con:

```
git config --global --list
```

También debes saber que Git maneja varios niveles de configuración:

- **sistema**;
- **global**;
- **local**.

La configuración local de un repositorio puede sobrescribir la global dentro de ese proyecto concreto.

> ⚠ **Precisión importante**
>
> La configuración global no tiene prioridad absoluta. Si un repositorio tiene una configuración local específica, esa configuración será la que se utilice dentro de ese proyecto.

Referencias del subtema

- Documentación oficial: `git config`.
- Git Book: personalización de Git.
- Git Cheat Sheet oficial.

---

3. El modelo mental correcto de Git

Este es el punto más importante de la sesión. Si entiendes bien este modelo, los comandos dejan de parecer arbitrarios y empiezan a tener sentido estructural.

En esta fase te conviene distinguir con claridad cinco espacios o niveles:

- **workspace** o directorio de trabajo;
- **index** o staging area;
- **repositorio local**;
- **stash**;
- **repositorio remoto** como anticipación de sesiones posteriores.

---

4. Workspace: tu directorio de trabajo

El **workspace** es la carpeta real del proyecto. Es el lugar donde editas archivos, escribes texto, cambias nombres, corriges errores o creas nuevos recursos.

Todo cambio empieza aquí.

4.1. Qué tipo de cambios viven en el workspace

En el workspace puedes tener:

- archivos nuevos;
- archivos modificados;
- archivos eliminados;
- trabajo incompleto;
- pruebas temporales;
- errores todavía no revisados.

Nada de eso forma parte automáticamente del historial.

4.2. Por qué debes distinguirlo bien

Si no distingues el workspace del resto del flujo, puedes llegar a pensar que “modificar” equivale a “guardar en Git”, y eso es incorrecto. Modificar solo significa alterar el estado de los archivos en tu directorio de trabajo.

> **Regla mental útil**
>
> El workspace es donde trabajas. No es todavía el historial.

Referencias del subtema

- Git Book: ciclo de vida de los archivos.
- Documentación oficial de Git.
- Explain Git with D3.

---

5. Index o staging area: la zona de preparación

El **index**, también llamado **staging area**, es una zona intermedia donde seleccionas qué cambios quieres incluir en el siguiente commit.

5.1. Por qué existe esta fase intermedia

Git no te obliga a confirmar todo lo que has modificado. Puedes preparar solo una parte del trabajo. Eso te permite construir commits más claros, más pequeños y más útiles.

5.2. Qué significa realmente “añadir a staging”

Cuando usas `git add`, no estás haciendo un simple gesto formal. Estás diciendo a Git: “quiero que esta versión actual del archivo forme parte del próximo commit”.

Por eso debes entender que:

- el workspace contiene cambios activos;
- el staging contiene cambios preparados;
- el repositorio local contiene cambios ya confirmados.

5.3. Por qué el staging es importante profesionalmente

La existencia del staging te permite:

- separar tareas distintas;
- evitar commits gigantes;
- controlar mejor el contenido del historial;
- revisar con más precisión lo que vas a confirmar.

> **Idea profesional**
>
> El buen historial no empieza en el commit. Empieza en cómo preparas los cambios en staging.

Referencias del subtema

- Documentación oficial: `git add`.
- Git Book: staging area.
- Learn Git Branching.

---

6. Repositorio local: el historial confirmado

El **repositorio local** contiene el historial de commits de tu proyecto en tu propia máquina.

Cada commit representa un estado registrado del proyecto con una intención concreta.

6.1. Qué entra en el repositorio local

Solo entran en el repositorio local los cambios que:

1. has preparado previamente;
2. has confirmado explícitamente con un commit.

6.2. Qué valor tiene el historial local

El historial local te permite:

- reconstruir la evolución del proyecto;
- volver atrás si es necesario;
- comparar estados;
- entender qué hiciste y en qué orden;
- preparar trabajo antes de compartirlo con un remoto.

> **Idea clave**

> El repositorio local ya es Git funcionando plenamente, aunque todavía no hayas trabajado con GitHub ni con remotos.

Referencias del subtema

- Git Book: snapshots y commits.
- Documentación oficial: `git commit`.
- Git Cheat Sheet oficial.

---

7. `stash`: almacenamiento temporal fuera del flujo principal

`stash` es un mecanismo que te permite apartar temporalmente cambios no confirmados para dejar el workspace limpio sin perderlos.

7.1. Qué problema resuelve `stash`

A veces puedes encontrarte en una situación como esta:

- estás trabajando en algo;
- todavía no quieres hacer commit;
- necesitas cambiar de contexto;
- te conviene dejar el directorio limpio.

En ese caso, `stash` te permite guardar temporalmente ese trabajo para recuperarlo después.

7.2. Qué no debes confundir

`stash` no es:

- un commit normal;
- una rama;
- el historial principal del proyecto;
- el repositorio remoto.

Es un almacenamiento temporal de trabajo en curso.

7.3. Diferencia entre workspace, index, repositorio y stash

Conviene que fijes esta comparación:

- **workspace**: donde editas;
- **index**: donde preparas;
- **repositorio local**: donde confirmas historial;
- **stash**: donde aparcas temporalmente trabajo sin convertirlo todavía en commit.

> ⚠ **Precaución conceptual**
>
> No uses `stash` como excusa para trabajar siempre de forma desordenada. Su utilidad está en gestionar interrupciones o cambios de contexto, no en sustituir una buena organización de commits y ramas.

Referencias del subtema

- Documentación oficial: `git stash`.
- Git Book: stashing y limpieza.
- Git Cheat Sheet oficial.

---

8. El repositorio remoto como anticipación

En esta sesión el foco está en el trabajo local, pero te conviene empezar a situar el repositorio remoto dentro del mapa mental general.

8.1. Qué es un remoto, de forma anticipada

Un **repositorio remoto** es otra copia del repositorio, alojada fuera de tu máquina y accesible normalmente por red.

8.2. Relación con lo que estás aprendiendo hoy

En esta sesión todavía no vas a trabajar en profundidad con `push`, `pull` o `fetch`, pero sí debes entender que el remoto no sustituye al repositorio local. Simplemente amplía el flujo hacia un contexto compartido.

> **Mapa general**
>
> Piensa el flujo así:
> workspace -> index -> repositorio local -> remoto

Referencias del subtema

- Git Book: repositorios remotos.
- Documentación oficial de Git.
- GitHub Docs.

---

9. Crear un repositorio local con `git init`

Para empezar a trabajar con Git en una carpeta, debes inicializarla como repositorio.

```
git init
```

Este comando crea la estructura interna necesaria para que Git pueda registrar historial.

9.1. Qué ocurre internamente

Al ejecutar `git init`, Git crea una carpeta oculta llamada `.git`. Ahí se guardan:

- configuración local;
- referencias;
- metadatos;
- historial del repositorio.

9.2. Cuándo usar `git init`

Usarás `git init` cuando:

- empieces un proyecto desde cero;
- quieras empezar a versionar una carpeta existente;
- necesites crear un repositorio local para practicar.

> ⚠ **No elimines `.git` sin saber lo que haces**
>
> La carpeta `.git` contiene la identidad del repositorio. Si la eliminas, el proyecto deja de estar versionado y pierdes su vínculo con el historial local.

Referencias del subtema

- Documentación oficial: `git init`.
- Git Book: creación de repositorios.
- Git Cheat Sheet oficial.

10. `README.md` y Markdown como primera documentación del proyecto

En proyectos reales, uno de los primeros archivos relevantes suele ser `README.md`.

10.1. Qué es `README.md`

`README.md` es, por convención, el archivo que presenta el proyecto. Suele incluir:

- nombre;
- descripción;
- propósito;
- instrucciones de uso;
- notas técnicas;
- autoría o contexto.

10.2. Por qué tiene sentido introducirlo desde el principio

Trabajar con Git no significa solo versionar código. También versionas documentación. Empezar con un `README.md` te permite practicar Git sobre contenido sencillo, legible y útil.

10.3. Qué es Markdown

Markdown es un lenguaje de marcado ligero orientado a documentación legible y fácil de escribir. Se utiliza mucho en repositorios porque permite redactar texto estructurado sin la complejidad de HTML.

10.4. Elementos básicos de Markdown

En esta fase te interesa reconocer al menos:

- títulos con `#`;
- listas con `-`;
- énfasis;
- enlaces;
- bloques de código;
- fragmentos en línea con comillas invertidas.

Ejemplo conceptual de estructura de `README.md`:

- título del proyecto;
- breve descripción;

- lista de contenidos o características;
- instrucciones iniciales.

> **ⓘ Idea importante**
>
> Markdown no sustituye a HTML en general, pero sí es una herramienta excelente para documentación técnica y archivos como `README.md`.

Referencias del subtema

- GitHub Docs: Markdown.
- Documentación general de Markdown.
- Buenas prácticas de README en repositorios.

---

11. Revisar el estado del proyecto con `git status`

`git status` es uno de los comandos más importantes al empezar.

```
git status
```

Te muestra qué está ocurriendo en el repositorio en cada momento.

11.1. Qué información te ofrece

Con `git status` puedes ver:

- archivos no rastreados;
- archivos modificados;
- archivos preparados en staging;
- rama actual;
- situaciones especiales de merge o conflicto.

11.2. Por qué debes consultarlo con frecuencia

Consultar `git status` te ayuda a no trabajar a ciegas. Gracias a este comando puedes saber:

- qué has cambiado;
- qué has preparado;
- qué te falta revisar;

- si el repositorio está limpio o no.

> **Hábito recomendado**
>
> Acostúmbrate a consultar `git status` antes de añadir, antes de hacer commit y después de hacer commit.

Referencias del subtema

- Documentación oficial: `git status`.
- Git Book: ciclo de vida de archivos.
- Git Cheat Sheet oficial.

---

12. Preparar cambios con `git add`

`git add` mueve cambios desde el workspace al index o staging area.

Ejemplos habituales:

```
git add README.md
git add notas.md
git add .
```

12.1. Qué significa realmente

Cuando haces `git add`, Git prepara el contenido actual del archivo para el siguiente commit.

12.2. Por qué no conviene abusar de `git add .`

`git add .` puede ser útil, pero no debería convertirse en un gesto automático desde el primer día.

Si lo usas sin revisar, podrías incluir:

- archivos temporales;
- cambios incompletos;
- recursos que no pertenecen a la tarea;
- archivos que todavía no quieres confirmar.

12.3. `git add` como selección consciente

Un buen uso de Git implica decidir qué cambios deben pasar al siguiente commit y cuáles no.

> **Criterio técnico**
>
> No todo lo que has modificado debe entrar en el mismo commit.

Referencias del subtema

- Documentación oficial: `git add`.
- Git Book: staging de archivos.
- Learn Git Branching.

---

13. Comparar cambios con `git diff`

`git diff` te permite inspeccionar diferencias entre estados del proyecto.

```
git diff
```

13.1. Para qué te sirve

Te ayuda a:

- revisar exactamente qué líneas han cambiado;
- detectar errores antes de confirmar;
- entender mejor el contenido real de tus modificaciones.

13.2. Relación con `git status`

`git status` te dice **qué archivos** están afectados.
`git diff` te enseña **qué ha cambiado dentro** de esos archivos.

> **Flujo recomendado**
>
> Antes de confirmar un commit, consulta primero `git status` y después `git diff`.

Referencias del subtema

- Documentación oficial: `git diff`.
- Git Book: inspección de cambios.

- Explain Git with D3.

---

14. Registrar cambios con `git commit`

El commit incorpora al repositorio local los cambios preparados en staging.

Ejemplo básico:

```
git commit -m "Añade README inicial del proyecto"
```

14.1. Qué representa un commit

Un commit no es simplemente “guardar”. Es una unidad de historial que registra una intención concreta.

14.2. Qué debe aportar un buen commit

Un buen commit suele ser:

- claro;
- específico;
- limitado a una intención principal;
- comprensible al leer el historial.

14.3. Qué debes evitar

Conviene evitar mensajes como:

- `cambios`
- `prueba`
- `cosas`
- `arreglo`
- `commit 2`

Ese tipo de mensajes debilita el valor del historial.

> **Criterio práctico**
>
> Si no puedes explicar en una frase qué contiene el commit, probablemente estás mezclando demasiado trabajo en él.

Referencias del subtema

- Documentación oficial: `git commit`.
- Git Book: registro de cambios.
- Buenas prácticas de mensajes de commit.

---

15. Conventional Commits como convención básica

En este bloque te conviene empezar a usar una versión simplificada de **Conventional Commits**.

15.1. Qué son

Conventional Commits es una convención para escribir mensajes de commit con una estructura reconocible. Su ventaja principal es que hace el historial más legible y más consistente.

15.2. Prefijos básicos que te interesa usar

En esta fase trabajarás sobre todo con:

- `feat:` para una nueva funcionalidad o incorporación relevante;
- `fix:` para una corrección;
- `docs:` para cambios de documentación;
- `refactor:` para reorganización interna sin cambiar el comportamiento esperado;
- `chore:` para tareas auxiliares o de mantenimiento.

15.3. Ejemplos

- `feat: añade estructura inicial del README`
- `docs: amplía instrucciones de uso`
- `fix: corrige errata en notas del proyecto`
- `chore: añade archivo gitignore base`

15.4. Por qué te interesa adoptarlos desde el principio

Si empiezas a trabajar con una convención clara desde las primeras sesiones, el historial será más fácil de leer, revisar y mantener cuando el proyecto crezca.

✓ **Hábito profesional**

Un commit bien escrito comunica intención antes incluso de abrir los archivos modificados.

Referencias del subtema

- Conventional Commits.
- Buenas prácticas de mensajes de commit.
- Recursos de equipos de desarrollo.

---

16. `git commit -am`: utilidad y límite

Git permite hacer commits de forma abreviada en algunos casos con:

```
git commit -am "mensaje"
```

16.1. Qué hace

Este comando:

- añade automáticamente a staging los cambios de archivos ya rastreados;
- realiza el commit en una sola operación.

16.2. Qué no hace

No añade archivos nuevos no rastreados.

Ese es el punto crítico: si has creado un archivo nuevo, `git commit -am` no lo incluirá automáticamente.

16.3. Cuándo puede resultarte útil

Puede ahorrarte tiempo en cambios pequeños sobre archivos ya conocidos por Git, pero no conviene usarlo sin pensar.

> ⚠ **Precaución**
>
> `git commit -am` no sustituye la necesidad de revisar el estado del repositorio. Úsalo solo cuando entiendas perfectamente qué archivos se van a incluir y cuáles no.

Referencias del subtema

- Documentación oficial: `git commit`.
- Git Book.
- Git Cheat Sheet oficial.

---

17. `git commit --amend`: corregir el último commit

Git permite modificar el último commit con:

```
git commit --amend
```

17.1. Para qué sirve

Te resulta útil cuando necesitas:

- corregir el mensaje del último commit;
- añadir algo pequeño que olvidaste incluir;
- ajustar inmediatamente el último paso del historial local.

17.2. Cuándo tiene sentido usarlo

Es especialmente útil cuando acabas de hacer un commit y detectas un error enseguida.

17.3. Precaución importante

`--amend` reescribe el último commit. En historial local es muy útil. Más adelante verás que, en contextos compartidos, reescribir historial exige más cuidado.

> **Criterio de uso**
>
> `git commit --amend` es una herramienta para corregir el último commit con rapidez, no para ocultar un flujo de trabajo desordenado de forma constante.

Referencias del subtema

- Documentación oficial: `git commit`.
- Git Book: reescritura del último commit.
- Recursos de buenas prácticas en Git.

18. Consultar el historial con `git log`

`git log` permite ver el historial del repositorio.

```
git log
```

También es muy útil la vista resumida:

```
git log --oneline
```

18.1. Qué información te aporta

Con `git log` puedes ver:

- identificadores de commit;
- autoría;
- fecha;
- mensajes de commit.

18.2. Por qué debes acostumbrarte a leer el historial

Trabajar con Git no consiste solo en producir commits, sino también en interpretarlos. Un historial claro te permite entender la evolución técnica del proyecto.

Referencias del subtema

- Documentación oficial: `git log`.
- Git Book: visualización del historial.
- Git Cheat Sheet oficial.

---

19. Ignorar archivos con `.gitignore`

No todo archivo de una carpeta debe formar parte del repositorio.

19.1. Qué suele interesarte ignorar

Es habitual ignorar:

- logs;

- cachés;
- archivos temporales;
- credenciales;
- configuraciones locales del editor;
- archivos del sistema operativo.

19.2. Por qué debes introducirlo pronto

Si no filtras bien qué entra en Git, el repositorio empieza a llenarse de ruido y eso complica el trabajo posterior.

19.3. Relación con la calidad del proyecto

Un repositorio limpio:

- comunica mejores prácticas;
- reduce errores;
- evita exponer información sensible;
- facilita el trabajo futuro.

Referencias del subtema

- Documentación oficial: `.gitignore`.
- Git Book: archivos ignorados.
- Plantillas comunes de `.gitignore`.

---

20. `git alias`: atajos para comandos frecuentes

Git permite crear alias para abreviar comandos largos o habituales.

20.1. Para qué sirven

Un alias te ayuda a trabajar con más fluidez cuando ya conoces bien el comando que estás acortando.

20.2. Ejemplos habituales

Por ejemplo, puedes crear un alias para una vista abreviada del historial o para una variante más rica de `log`.

Muchos equipos crean alias personalizados para comandos de historial, estado o revisión. Un caso típico es definir alias de `log --oneline --decorate --all --graph` o variantes

parecidas.

20.3. Criterio de uso

Los alias mejoran ergonomía, pero no deben sustituir tu comprensión del comando original.

> **Regla práctica**
>
> Primero entiende el comando real. Después, si lo usas mucho, considera crear un alias.

Referencias del subtema

- Documentación oficial: `git config`.
- Recursos de productividad con Git.
- Chuletas avanzadas de Git.

---

21. Primeras ramas: por qué las necesitas ya

Aunque profundizarás más adelante, tiene sentido introducir ramas desde esta misma sesión.

21.1. Qué es una rama

Una rama es una línea de trabajo dentro del mismo repositorio. Te permite desarrollar cambios de forma separada sin alterar directamente otra línea del historial.

21.2. Qué problema resuelven

Las ramas te ayudan a:

- separar tareas;
- experimentar con menos riesgo;
- organizar mejor el trabajo;
- preparar integraciones posteriores.

21.3. Por qué conviene introducirlas pronto

Si aprendes Git solo como una secuencia lineal de commits sobre una única rama, después te costará más entender los flujos reales de trabajo.

> **Idea importante**

> Trabajar con ramas no es una rareza avanzada. Es parte normal del uso real de Git.

Referencias del subtema

- Git Book: ramas.
- Documentación oficial: `git branch`.
- Learn Git Branching.

---

22. Ver y crear ramas con `git branch`

Puedes consultar las ramas disponibles con:

```
git branch
```

Y crear una nueva rama con:

```
git branch nombre-de-la-rama
```

22.1. Qué debes recordar

Crear una rama no significa cambiarte automáticamente a ella.

22.2. Nombrado básico de ramas

Desde el principio te conviene usar nombres claros, por ejemplo:

- `feature/readme-ampliado`
- `docs/notas-iniciales`
- `fix/correccion-bienvenida`

Referencias del subtema

- Documentación oficial: `git branch`.
- Git Book: creación de ramas.
- Git Cheat Sheet oficial.

---

23. Cambiar de rama: `git switch` y `git checkout`

Hoy en día, Git ofrece `git switch` como comando más específico para cambiar de rama.

```
git switch nombre-de-la-rama
```

También puedes crear y cambiar en un solo paso:

```
git switch -c nombre-de-la-rama
```

23.1. El papel de `git checkout`

Históricamente, `git checkout` se ha usado para cambiar de rama y para otras operaciones. Por eso lo vas a encontrar constantemente en tutoriales, documentación antigua y proyectos existentes.

Ejemplo:

```
git checkout nombre-de-la-rama
```

23.2. Cómo debes interpretarlo

En este bloque te conviene pensar así:

- `git switch` como comando principal para cambiar de rama;
- `git checkout` como comando importante que debes reconocer, aunque tenga usos más amplios y más antiguos.

> ⚠ **Precisión práctica**
>
> Aunque te acostumbres a `git switch`, no puedes ignorar `git checkout`, porque seguirá apareciendo en muchos materiales reales.

Referencias del subtema

- Documentación oficial: `git switch`.
- Documentación oficial: `git checkout`.
- Git Book.

---

24. Primer contacto con `merge`

Una vez que has creado una rama y has trabajado en ella, en algún momento necesitarás integrar sus cambios con otra rama.

24.1. Qué hace `merge`

`git merge` une el historial de una rama dentro de otra.

24.2. Idea operativa básica

El flujo conceptual es:

1. trabajas en una rama;
2. haces commits allí;
3. vuelves a la rama de destino;
4. integras con `merge`.

24.3. Por qué se introduce ya aquí

No se trata de profundizar todavía en conflictos complejos, pero sí de que empieces a ver que Git no es solo una línea única de commits.

Referencias del subtema

- Documentación oficial: `git merge`.
- Git Book: fusiones básicas.
- Learn Git Branching.

---

25. Marcadores de conflicto: qué son y por qué debes reconocerlos

Cuando Git no puede resolver automáticamente una integración, inserta marcas visibles dentro del archivo en conflicto.

25.1. Qué debes reconocer

Esas marcas delimitan:

- una versión de una línea de trabajo;
- otra versión en conflicto;
- el lugar exacto donde Git necesita una decisión humana.

25.2. Por qué es importante mencionarlo ya

Aunque la resolución detallada de conflictos se trabaje con más profundidad después, necesitas reconocer desde ahora que un conflicto no se manifiesta solo como un mensaje en terminal, sino también como una estructura visible dentro del archivo afectado.

25.3. Qué no debes hacer

No debes dejar esos marcadores en el archivo final. Son una ayuda temporal para resolver el conflicto, no contenido válido del proyecto.

> ⚠ **Idea clave**
>
> Si ves marcadores de conflicto en un archivo, Git te está diciendo que todavía no has terminado la integración correctamente.

Referencias del subtema

- Git Book: conflictos de merge.
- Documentación oficial de Git.
- Learn Git Branching.

---

26. `git reset`, `git restore` y el cuidado al deshacer

Git ofrece varias formas de deshacer, pero no todas tienen el mismo efecto ni el mismo riesgo.

26.1. `git restore`

Te ayuda a restaurar archivos del workspace o del staging según el caso. Es una forma más explícita y moderna de trabajar con ciertos escenarios de reversión.

26.2. `git reset`

`git reset` puede utilizarse para distintos fines, entre ellos mover cambios fuera del staging o alterar referencias del historial en ciertos contextos.

26.3. Por qué debes ser prudente

No todo comando para “deshacer” es inocuo. Algunos afectan solo al staging; otros pueden afectar al historial; otros pueden eliminar trabajo no confirmado si se usan mal.

> ⚡ **Criterio de seguridad**

> Antes de usar un comando para deshacer, debes saber si afecta al workspace, al index o al historial. No uses `reset` como reflejo automático.

Referencias del subtema

- Documentación oficial: `git restore`.
- Documentación oficial: `git reset`.
- Git Book: deshacer cosas.

---

27. Flujo local recomendado para esta sesión

A partir de todo lo anterior, el flujo que vas a practicar en esta sesión puede resumirse así:

27.1. Flujo base de trabajo local

1. Creas o abres un repositorio con `git init`.
2. Trabajas en archivos como `README.md`, notas o documentación.
3. Compruebas el estado con `git status`.
4. Revisas cambios con `git diff`.
5. Preparas solo lo que corresponda con `git add`.
6. Confirmas con `git commit`.
7. Lees el historial con `git log`.
8. Cuando una tarea lo requiere, abres una rama.
9. Trabajas en ella y haces commits con intención.
10. Vuelves a la rama de destino e integras con `merge` en situaciones básicas.

27.2. Hábitos que te interesa consolidar

Durante esta sesión debes intentar fijar estos hábitos:

- no trabajar a ciegas;
- consultar el estado con frecuencia;
- no abusar de `git add .`;
- escribir commits útiles;
- usar una convención básica de Conventional Commits;
- entender la diferencia entre workspace, index, historial local y stash;
- abrir ramas cuando la tarea lo justifique;

- reconocer `switch` y `checkout`.

> ✓ **Competencia que debes adquirir**
>
> Al terminar esta sesión deberías ser capaz de crear un repositorio local, documentarlo con un `README.md`, trabajar con Markdown básico, preparar cambios conscientemente, hacer commits claros, usar ramas sencillas y explicar la arquitectura mental básica de Git.

---

28. Análisis técnico avanzado

Esta sección amplía lo anterior y te ayuda a no quedarte en una visión superficial de Git.

28.1. Git no registra simplemente “archivos”: registra estados

Cada commit representa un estado del proyecto, no solo una acumulación informal de archivos tocados. Esta idea es importante porque explica por qué el historial tiene estructura y por qué los commits deben responder a una intención.

28.2. El staging no es una complicación innecesaria

El index existe precisamente para darte control. Cuanto mejor entiendas este paso, mejor será la calidad de tu historial. Si ignoras conceptualmente el staging, terminarás haciendo commits caóticos y poco útiles.

28.3. `stash` no pertenece al flujo principal del historial

`stash` es útil porque te deja aparcar trabajo, pero no sustituye al historial ni a las ramas. Debes verlo como mecanismo auxiliar y temporal.

28.4. Conventional Commits no son solo estética

Una convención clara de mensajes mejora la lectura del historial, la revisión y la comunicación técnica. Incluso en proyectos pequeños, esa disciplina compensa.

28.5. `checkout` sigue siendo relevante aunque uses `switch`

Muchos equipos, tutoriales y repositorios siguen usando `checkout`. Por eso necesitas reconocerlo, aunque pedagógicamente sea más claro separar funciones con `switch` y `restore`.

28.6. Las ramas no son una ampliación exótica

Introducir ramas desde la primera sesión fuerte de Git evita que construyas una imagen falsa del trabajo real. Git no está pensado solo para una línea única de commits.

28.7. Limitaciones de esta sesión

En esta sesión todavía no profundizas en:

- remotos;
- sincronización con GitHub;
- Pull Requests;
- conflictos complejos;
- `rebase`, `revert`, `cherry-pick` o `bisect` como herramientas de trabajo posterior.

Esto es intencional. Primero necesitas construir una base sólida de trabajo local.

---

29. Síntesis final

En esta sesión has trabajado el núcleo conceptual de Git: has diferenciado el workspace, el index, el repositorio local, `stash` y la futura capa remota; has comprendido que Git no guarda automáticamente lo que haces, sino que te obliga a seleccionar y confirmar cambios con intención.

También has empezado a utilizar `README.md` y Markdown como primera documentación del proyecto, has introducido una convención básica de Conventional Commits y has dado los primeros pasos con ramas, `switch`, `checkout` y `merge`.

La idea central que debes conservar es esta: Git no es una lista de comandos para memorizar, sino una estructura de trabajo. Cuanto mejor entiendas esa estructura, más fácil te resultará avanzar después hacia remotos, colaboración y flujos más complejos.

En la siguiente sesión trasladarás esta base al trabajo con **repositorios remotos, GitHub y sincronización**, donde el historial local empezará a relacionarse con un entorno compartido.

Referencias generales

Documentación oficial y manuales

- Git Book en español: `https://git-scm.com/book/es/v2`
- Documentación oficial de Git: `https://git-scm.com/docs/git/es`

Documentación específica de apoyo

- Conventional Commits: `https://www.conventionalcommits.org/`
- Documentación de GitHub sobre Markdown: `https://docs.github.com/es/get-started/writing-on-github`

Recursos visuales y de consulta

- Git Cheat Sheet interactiva: `https://www.ndpsoftware.com/git-cheatsheet.html`
- Explain Git with D3: `https://onlywei.github.io/explain-git-with-d3/`

Práctica recomendada

- Learn Git Branching: `https://learngitbranching.js.org/?locale=es_ES`
- Git Exercises: `https://gitexercises.fracz.com/`

Materiales de apoyo complementarios

- Chuleta de Git de Atlassian.
- GitHub Git Cheat Sheet.